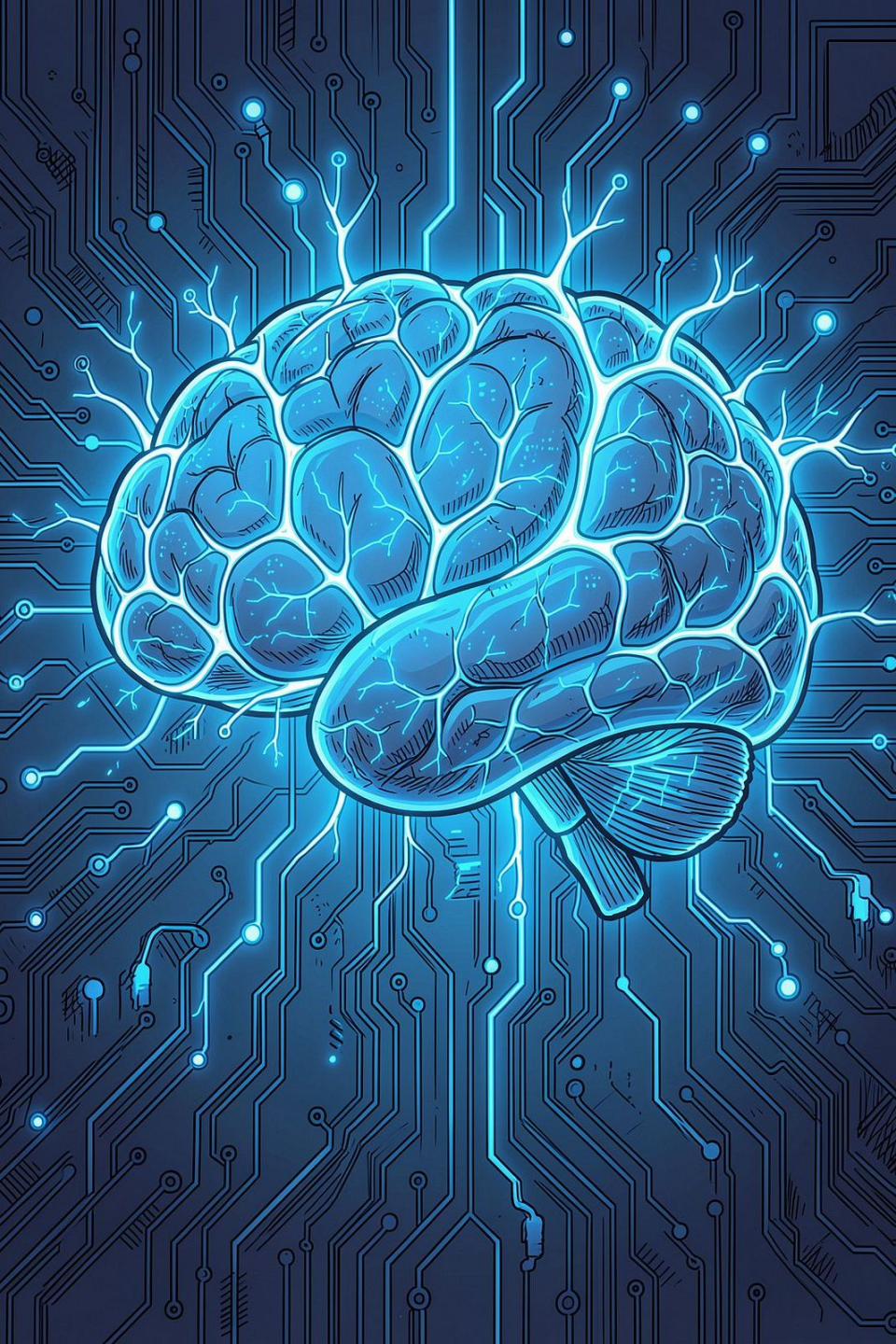




Teoria e Aplicações de Inteligência Computacional I

Unidade 1 – Aula 2 – Introdução aos Fundamentos de
Inteligência Computacional

Prof. Dr. Guilherme Augusto Defalque

Mestrado em Engenharia Elétrica - PPGEE - UFMS

Conceitos e paradigmas de redes neurais

Agenda da Aula



Cap. 1 — Fundamentos da IC

IA por Regras · ML · Deep Learning



Cap. 2 — Aprendizado de Máquina

Datasets · Métricas · Overfitting



Cap. 3 — Aprendizado Supervisionado

Classificação · Regressão · Eng. Elétrica



Cap. 4 — Aprendizado Não Supervisionado

K-Means · PCA



Cap. 5 — Redes Neurais Artificiais

Perceptron · MLP · Sigmoid



Cap. 6 — Treinamento de Redes

Backpropagation · Gradiente Descendente



Capítulo 1

Fundamentos da Inteligência Computacional

Da lógica formal ao aprendizado a partir de dados.

IA Clássica: A Abordagem por Regras

1 Primórdios da IA

Solucionar problemas complexos para humanos, mas simples para computadores através de regras formais.

2 Base de Conhecimento

Codificação do mundo em linguagens formais para raciocínio lógico.

3 Problema Central

A complexidade do mundo real é impraticável de ser "hard-codificada".

4 Exemplo

Reconhecimento de fala ou rostos: trivial para humanos, impossível via regras.

5 Conclusão

Sistemas baseados em regras não escalam para situações do cotidiano.

A Evolução: ML e Deep Learning

1

1943 — McCulloch & Pitts

Primeiro modelo matemático de neurônio artificial. Base teórica para redes neurais.

2

1958 — Perceptron (Rosenblatt)

Frank Rosenblatt cria o Perceptron, primeiro algoritmo de aprendizado supervisionado.

3

1969 — Inverno da IA (Minsky & Papert)

Livro 'Perceptrons' demonstra limitações do modelo. Financiamento e pesquisa reduzidos drasticamente.

4

1986 — Backpropagation (Rumelhart et al.)

Algoritmo de retropropagação popularizado, permitindo treinar redes multicamadas (MLP).

5

1998 — LeNet (LeCun)

Yann LeCun aplica redes convolucionais ao reconhecimento de dígitos manuscritos (MNIST).

A Evolução: ML e Deep Learning (cont.)

Continuando nossa jornada pela história da inteligência artificial, exploramos os marcos que definiram a era moderna do Machine Learning e Deep Learning.

- 1 — 2006 — Deep Belief Networks (Hinton)**
Geoffrey Hinton ressuscita o Deep Learning com redes profundas pré-treinadas camada a camada.
- 2 — 2012 — AlexNet (ImageNet)**
AlexNet vence o ImageNet com GPU, marcando o início da era moderna do Deep Learning.
- 3 — 2017 — Transformers (Attention is All You Need)**
Arquitetura Transformer revoluciona NLP e abre caminho para GPT, BERT e LLMs.
- 4 — 2022–hoje — IA Generativa**
ChatGPT, Stable Diffusion e modelos multimodais transformam indústrias globalmente.

Dados, Dataset e Amostras



Dados

Registros de observações do mundo real ou processos físicos. Representação digital da experiência.

Em Engenharia Elétrica, frequentemente **séries temporais** (leitura horária de rendimento, potência em watts instantânea, etc.).



Dataset

Coleção de registros agrupados para uma finalidade específica. Funciona como repositório de exemplos.

Exemplo clássico: **dataset Iris** – 150 plantas de íris com 4 medições cada.



Amostra (Observação/Exemplo)

Unidade mínima de informação dentro de um dataset. Em termos estatísticos, é a realização de um vetor aleatório.

Atributos, Target e Matriz de Design



Atributos (Features)

Características que descrevem cada amostra. Ex: pixels de imagem, comprimento de sépalas (dataset Iris).



Variável Alvo (Target)

O que queremos prever ou classificar. Ex: espécie da planta, consumo de energia.



Vetor de Amostra

$x \in \mathbb{R}^n$: lista ordenada de n atributos para uma amostra.



Matriz de Design X

$X \in \mathbb{R}^{(m \times n)}$: m exemplos (linhas) $\times$ n atributos (colunas). Traduz dados em formato matricial.



Vetor de Rótulos y

Cada y_i fornece a categoria/valor do i -ésimo exemplo.



Exemplo MNIST

70.000 imagens de dígitos (0-9). Imagem = vetor de pixels. Dígito real = variável alvo.

Exemplo em Python: Carregando o Dataset Iris

O dataset Iris é um clássico para ilustrar atributos, target e matriz de design. Veja como carregá-lo e inspecioná-lo com scikit-learn e pandas.

```
import pandas as pd
from sklearn.datasets import load_iris

# Carregar o dataset Iris
iris = load_iris()

# Criar DataFrame com os atributos (features)
X = pd.DataFrame(iris.data, columns=iris.feature_names)

# Criar Series com o target (variável alvo)
y = pd.Series(iris.target, name='target')

# Visualizar as primeiras amostras
print("=== Matriz de Design X (primeiras 5 linhas) ===")
print(X.head())

print("\n=== Vetor de Rótulos y (primeiras 5 amostras) ===")
print(y.head())

print("\n=== Informações do Dataset ===")
print(f"Número de amostras (m): {X.shape[0]}")
print(f"Número de atributos (n): {X.shape[1]}")
print(f"Classes: {iris.target_names}")
```

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
5.1	3.5	1.4	0.2	0 (setosa)
4.9	3.0	1.4	0.2	0 (setosa)
4.7	3.2	1.3	0.2	0 (setosa)
4.6	3.1	1.5	0.2	0 (setosa)
5.0	3.6	1.4	0.2	0 (setosa)

m=150 amostras, n=4 atributos, 3 classes (setosa, versicolor, virginica)

Capítulo 2

A Mecânica do Aprendizado em Sistemas Inteligentes

Como sistemas adaptativos adquirem conhecimento a partir da experiência.



Definição Formal de Aprendizado de Máquina

"Diz-se que um programa de computador aprende com a experiência E em relação a alguma classe de tarefas T e medida de desempenho P , se o seu desempenho em tarefas em T , conforme medido por P , melhora com a experiência E ."

— Tom Mitchell

A definição de Mitchell estabelece a tríade fundamental do aprendizado de máquina:

Definição Formal de Aprendizado de Máquina

1 Tarefa (T)

A habilidade específica que o sistema deve adquirir ou o problema que precisa resolver.

Ex: Classificar falhas em linhas de transmissão, prever o preço de uma ação.

2 Desempenho (P)

Uma métrica quantitativa para avaliar o quão bem o sistema executa a tarefa.

Ex: Acurácia (para classificação), Erro Quadrático Médio (MSE para regressão).

3 Experiência (E)

Os dados aos quais o algoritmo é exposto e que utiliza para aprender.

Pode ser **Supervisionado** (com rótulos) ou **Não Supervisionado** (sem rótulos).

Treinamento, Predição e Erro

Treinamento

Ajuste dos parâmetros livres (pesos w e bias b) para mapear entradas em saídas desejadas.

Predição

Após treinamento, o modelo recebe entrada inédita e gera estimativa $\hat{y} = w^T x$.

Erro

Discrepância entre predição $\hat{y}$ e valor real y . Essencial para "direcionar" o ajuste do sistema.

Métricas de Erro e Otimização

MSE (Mean Squared Error)

Penaliza erros grandes ao elevar ao quadrado as diferenças. Base para outras métricas.

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

RMSE (Root Mean Squared Error)

Raiz quadrada do MSE, na mesma unidade da variável alvo. Penaliza erros grandes.

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2}$$

R² (Coeficiente de Determinação)

Mede a proporção da variância dos dados explicada pelo modelo. Varia de 0 a 1 (ou pode ser negativo para modelos ruins). R² = 1 indica ajuste perfeito.

$$R^2 = 1 - \frac{\sum_{i=1}^m (\hat{y}_i - y_i)^2}{\sum_{i=1}^m (y_i - \bar{y})^2}$$

Generalização e Divisão dos Dados

A capacidade de um modelo de performar bem em dados **NOVOS** é o verdadeiro objetivo do aprendizado.

Imagine um estudante que memoriza respostas de provas antigas, mas não consegue resolver problemas novos da matéria.

Treinamento

Ajuste efetivo dos parâmetros do modelo com a maior parte dos dados.

Validação

Monitoramento do desempenho e ajuste de hiperparâmetros (ex: taxa de aprendizado, complexidade).

Não influencia diretamente os pesos.

Teste

Dados completamente novos, nunca vistos pelo modelo.

A **única** medida confiável de generalização real.

Para datasets limitados, a **Validação Cruzada (k-fold)** divide os dados em 'k' partes, alternando qual serve como validação para um teste mais robusto.

Underfitting, Overfitting e o Trade-off Viés-Variância

Underfitting (Alto Viés)

Modelo muito simples.

Erro alto no treino E no teste.

Não captura a estrutura dos dados.

Análogo a representar parábola com linha reta.

Capacidade Ótima

Equilíbrio entre complexidade e simplicidade.

Generaliza bem.

Overfitting (Alta Variância)

Modelo muito complexo.

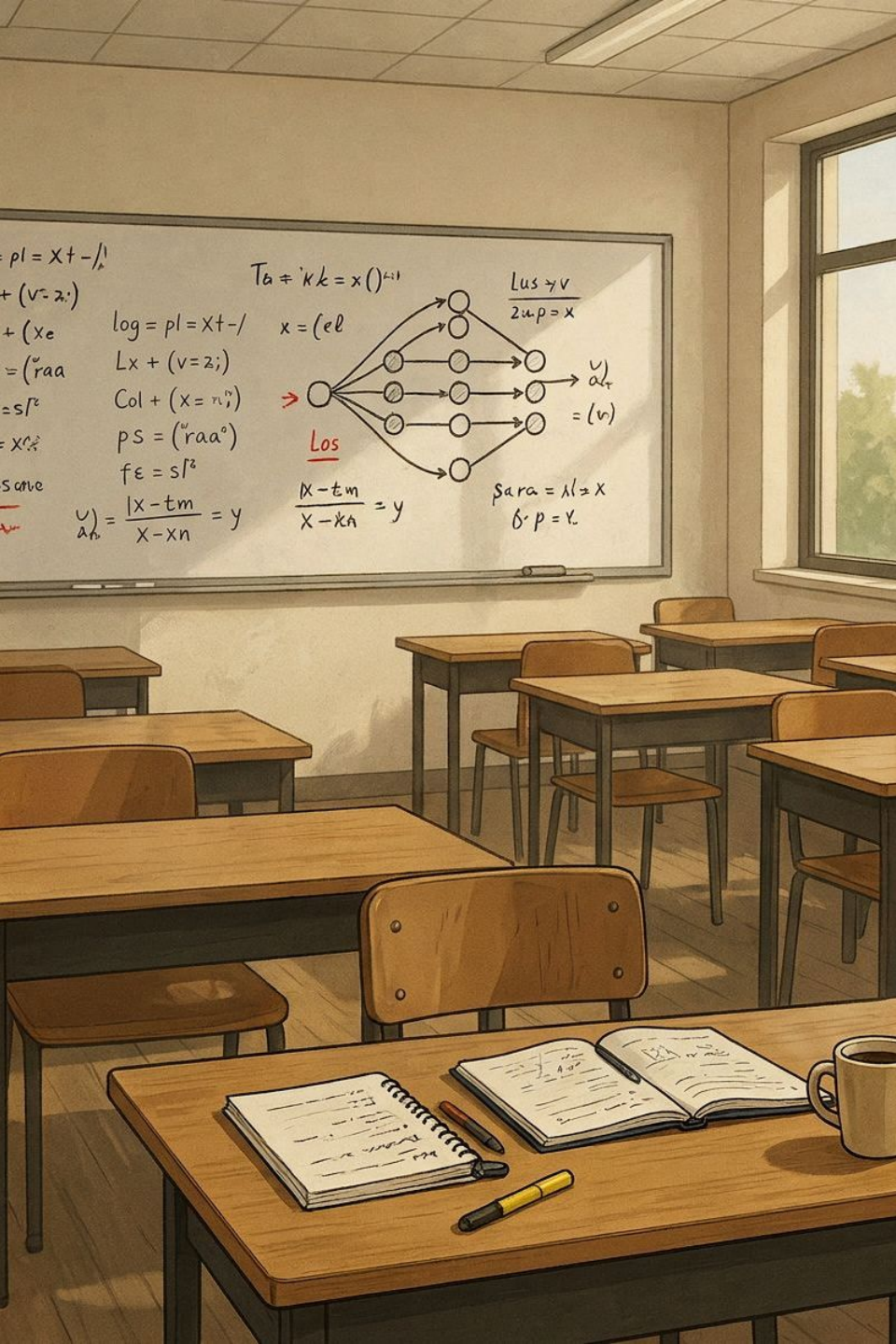
Erro baixo no treino, alto no teste.

"Decora" o ruído.

Trade-off Viés-Variância

Ao aumentar a complexidade do modelo, reduz-se o viés (subajuste) mas aumenta-se a variância (sobreajuste).

Regularização: introduz um pequeno viés para reduzir a variância significativamente, buscando um melhor equilíbrio.



$$p = x + \frac{1}{x}$$
$$+ (v = 2)$$
$$+ (x = e)$$
$$= (raa)$$
$$= sf^2$$
$$= x^2$$
$$= sf^2$$
$$u = \frac{x - tm}{x - xn} = y$$

$$\log = p = x + \frac{1}{x}$$
$$Lx + (v = 2)$$
$$Col + (x = r_{i,j})$$
$$ps = (raa^0)$$
$$f\epsilon = sf^2$$

$$Ta = 'kk = x^{(k)}$$
$$x = (el)$$

Los

$$\frac{x - tm}{x - xn} = y$$

$$\frac{Lus \gamma v}{2up = x}$$
$$u = (v)$$
$$sara = \lambda \gamma x$$
$$\delta \cdot p = \gamma$$

Capítulo 3

Aprendizado Supervisionado

Aprender com um professor: pares de entrada e resposta desejada.

Classificação vs. Regressão

Classificação

- Saída discreta (categorias). $f : R^n \rightarrow \{1, \dots, k\}$
- Traçar fronteiras de decisão no espaço multidimensional.
- Ex: identificar dígito manuscrito (0–9), Perceptron de Rosenblatt.
- Métricas: Acurácia, Precisão, Recall.

Regressão

- Saída contínua (valor numérico). $f : R^n \rightarrow R$
- Encontrar curva/superfície que melhor se ajusta aos dados.
- Ex: previsão de carga elétrica, $\hat{y} = w^T x + b$.
- Métricas: MSE, MAE, R^2 .

Aplicações em Engenharia Elétrica



Radar

Classificação de ecos (aeronaves, pássaros, clutter) utilizando modelos autorregressivos e redes neurais avançadas.



Controle

Identificação de sistemas dinâmicos não lineares, com redes neurais atuando como preditores de um passo à frente.



Sistemas de Potência

Previsão de demanda de carga (load forecasting) e análise de estabilidade para otimização da rede elétrica.



Telecomunicações

Equalização adaptativa de canais com algoritmos LMS, garantindo comunicação robusta em tempo real.



Reconhecimento de Padrões

OCR para leitura automatizada de cheques e análise de imagens térmicas para detecção de falhas em transformadores.

Capítulo 4

Aprendizado Não Supervisionado

Descobrimo estruturas ocultas nos dados — sem rótulos.



Clustering: O Algoritmo K-Means

Dividir dados em grupos onde membros são mais similares entre si do que com outros grupos.

K-Means

Técnica popular que escolhe K centróides e os ajusta iterativamente para formar clusters.

Função de Custo

$$J(C) = \frac{1}{m} \sum_{i=1}^m \sum_{x \in C_i} \|x - \mu_i\|^2$$

Minimiza a distância intra-cluster, buscando a compacidade dos grupos.

Processo Iterativo

- **Atribuição:** Cada amostra é atribuída ao centróide mais próximo (distância Euclidiana).
- **Atualização:** Cada centróide é recalculado como a média das amostras atribuídas a ele.

Quando Usar

Ideal para segmentar dados em categorias discretas e mutuamente exclusivas, sem rótulos prévios.

Aplicação em Engenharia Elétrica

Agrupamento de perfis de carga de consumidores para otimização de tarifas ou gestão de demanda.

Exemplo Numérico: K-Means com K=2

Considere 6 pontos 2D. Vamos executar o K-Means com K=2 manualmente, passo a passo.

01

Dados Iniciais:

Pontos: A=(1,1), B=(1,2), C=(2,1), D=(8,8), E=(8,9), F=(9,8)

Centróides iniciais (escolhidos aleatoriamente): $\mu_1=(1,1)$, $\mu_2=(8,8)$

02

Iteração 1: Atribuição

Calcular distância Euclidiana de cada ponto aos centróides:

$$d(x, \mu) = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2}$$

- $d(A, \mu_1) = 0,0 \rightarrow d(A, \mu_2) = 9,9 \rightarrow \text{Cluster 1}$
- $d(B, \mu_1) = 1,0 \rightarrow d(B, \mu_2) = 9,2 \rightarrow \text{Cluster 1}$
- $d(C, \mu_1) = 1,0 \rightarrow d(C, \mu_2) = 9,2 \rightarrow \text{Cluster 1}$
- $d(D, \mu_1) = 9,9 \rightarrow d(D, \mu_2) = 0,0 \rightarrow \text{Cluster 2}$
- $d(E, \mu_1) = 10,6 \rightarrow d(E, \mu_2) = 1,0 \rightarrow \text{Cluster 2}$
- $d(F, \mu_1) = 10,6 \rightarrow d(F, \mu_2) = 1,0 \rightarrow \text{Cluster 2}$

03

Iteração 1: Atualização dos Centróides

Cluster 1 = {A, B, C}:

$$\mu_1 = \left(\frac{1 + 1 + 2}{3}, \frac{1 + 2 + 1}{3} \right) = (1,33, 1,33)$$

Cluster 2 = {D, E, F}:

$$\mu_2 = \left(\frac{8 + 8 + 9}{3}, \frac{8 + 9 + 8}{3} \right) = (8,33, 8,33)$$

04

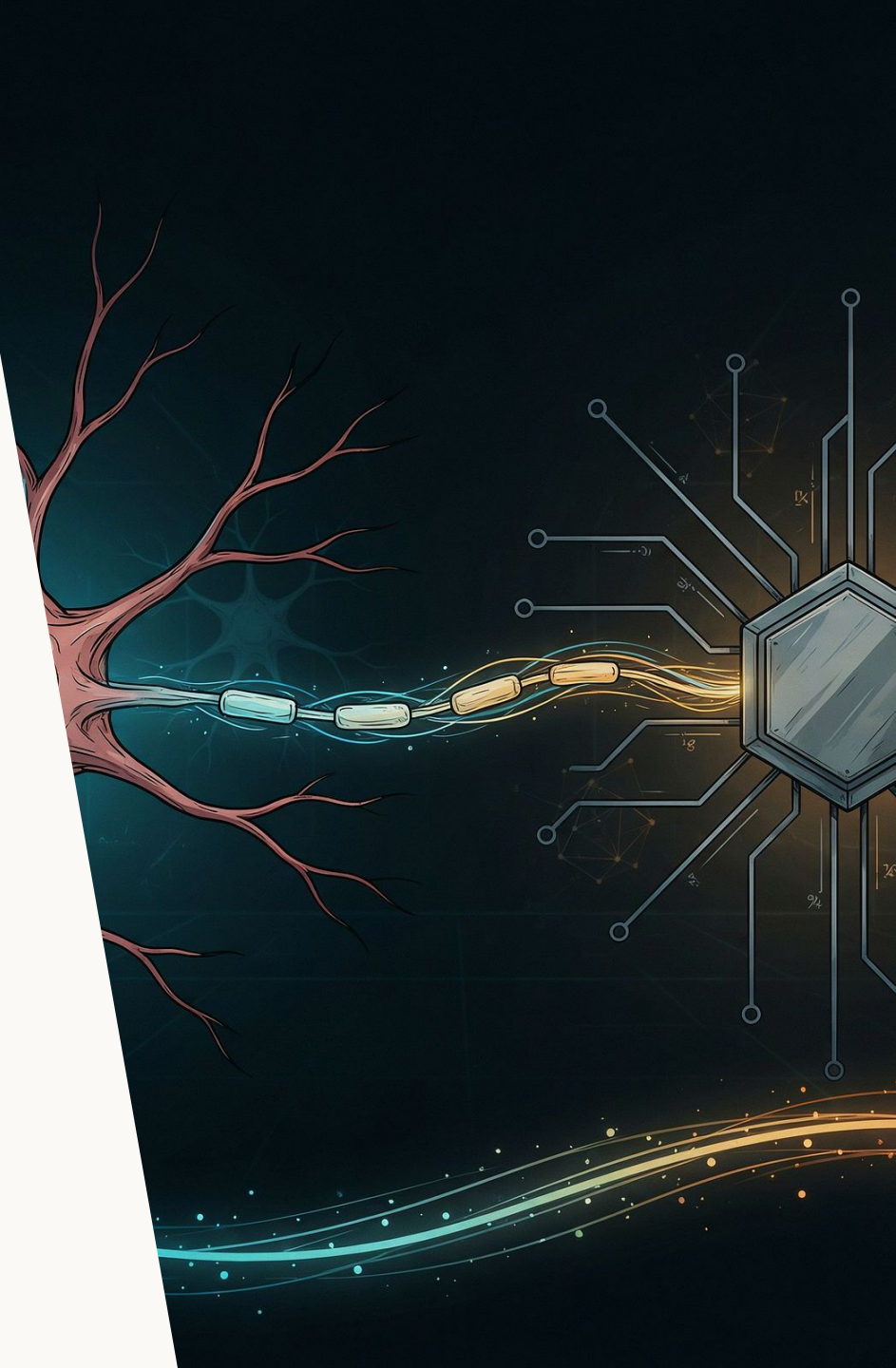
Iteração 2: Verificação de Convergência

Reatribuir pontos com os novos centróides $\mu_1=(1,33; 1,33)$ e $\mu_2=(8,33; 8,33)$.

Capítulo 5

O Neurônio Artificial

A unidade fundamental de processamento das Redes Neurais Artificiais.





Inspiração Biológica: O Neurônio Biológico

O cérebro humano, com aproximadamente **10 bilhões de neurônios** e **60 trilhões de sinapses**, opera com notável **eficiência energética** ($\sim 10^{-16}$ joules/operação). Sua **plasticidade cerebral** — a capacidade de criar e modificar conexões sinápticas — serve como análogo fundamental para o **aprendizado em Redes Neurais Artificiais**.



Dendritos

Zonas receptivas que recebem sinais de milhares de outras células. Equivalem às entradas ($x_1, x_2, \dots, x_n$) do neurônio artificial.



Corpo Celular (Soma)

Integra e processa todos os sinais recebidos pelos dendritos. Equivale à soma ponderada + bias: $v_k = \sum w_{kj} \cdot x_j + b_k$



Axônio

"Linha de transmissão" que propaga o sinal de saída do neurônio para outras células. Equivale à saída y_k do modelo.



Sinapses

Junções que convertem sinais elétricos em químicos (e vice-versa), modulando a força da conexão. Equivalem aos pesos sinápticos w_{kj} .



Potenciais de Ação (Spikes)

Pulsos de voltagem que disparam quando o estímulo ultrapassa um limiar. Equivalem à função de ativação $\phi(v_k)$, que decide se o neurônio "dispara".

Componentes do Neurônio Artificial

Cada neurônio artificial processa informações através de uma série de etapas bem definidas.

01

Entradas ($x_1, x_2, \dots, x_m$)

Sinais do ambiente ou de outras camadas. Ex: medições de tensão/corrente.

02

Pesos Sinápticos (w_{kj})

Força de cada conexão. Positivos = excitatórios. Negativos = inibitórios. Armazenam o conhecimento.

03

Bias (b_k)

Parâmetro externo que ajusta o "limiar de disparo". Tratado como peso extra $w_{k0} = b_k$ com entrada fixa $x_0 = +1$.

04

Soma Ponderada

$$u_k = \sum w_{kj}x_j$$

05

Campo Local Induzido

$$b_k + \sum w_{kj}x_j$$

(com $x_0 = 1$)

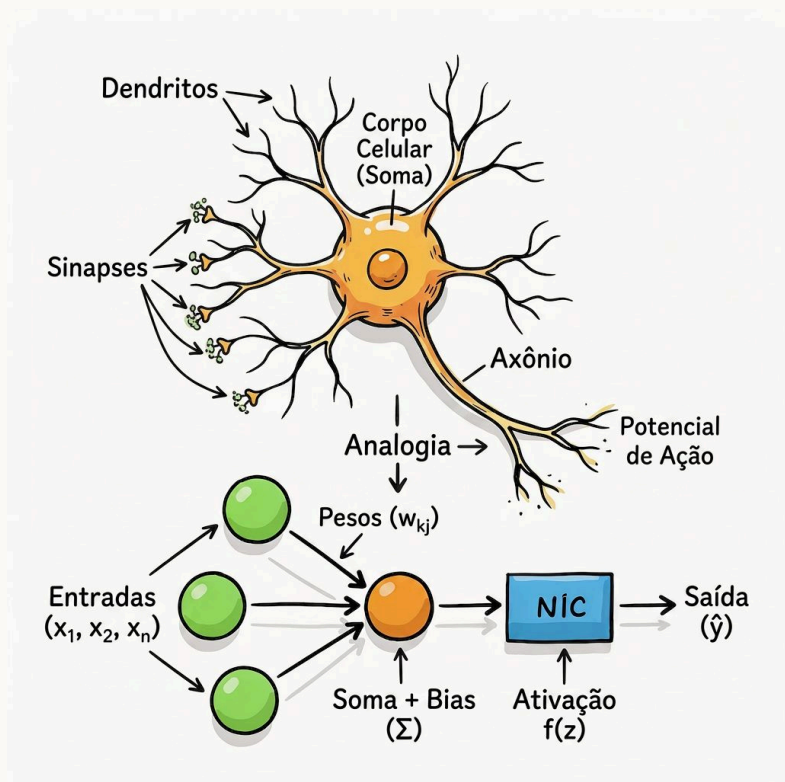
06

Função de Ativação

$$y_k = \varphi(v_k)$$

Limita a amplitude da saída e introduz não-linearidade.

Do Neurônio Biológico ao Artificial



Dendritos →
Entradas ($x_1 \dots x_n$)

Sinais recebidos do
ambiente.

Sinapses → Pesos
(w_{kj})

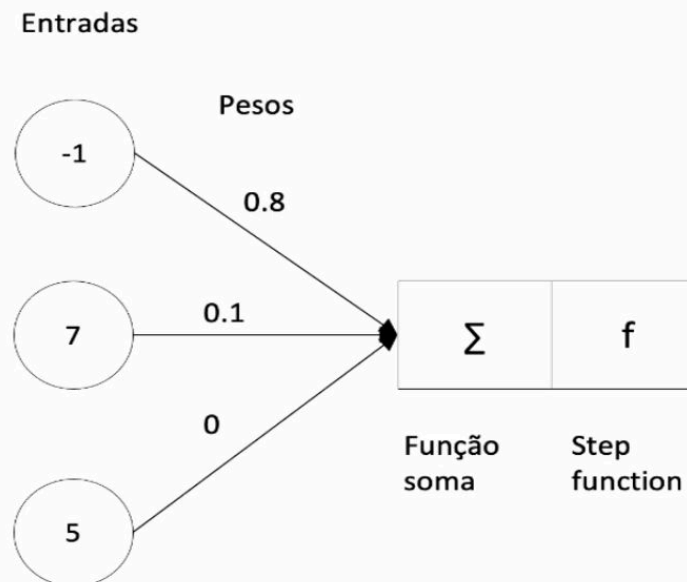
Força de cada
conexão.

Corpo Celular →
Soma + Bias (z)

$$v_k = \sum w_{kj} x_j + b_k$$

Potencial de Ação
→ Ativação $f(z)$

$$y_k = \varphi(v_k)$$



Axônio → Saída (y)

Propaga o resultado para a próxima camada.

Exemplo Numérico: Calculando um Neurônio Artificial

Vamos calcular manualmente a saída de um neurônio com 3 entradas, usando a função de ativação degrau.

01

Dados de Entrada

Entradas: $x_1 = 1$, $x_2 = 0$, $x_3 = 1$

Pesos: $w_1 = 0,5$ | $w_2 = -1,0$ | $w_3 = 0,8$

Bias: $b = -0,2$

02

Soma Ponderada (z)

Calcular a soma ponderada de cada entrada multiplicada pelo seu peso:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$z = (0,5 \times 1) + (-1,0 \times 0) + (0,8 \times 1) + (-0,2)$$

$$z = 0,5 + 0 + 0,8 - 0,2 = 1,1$$

03

Função de Ativação (Degrâu)

Aplicar a função degrau ao resultado $z = 1,1$:

$$f(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}$$

Como $z = 1,1 \geq 0 \rightarrow f(z) = 1$

04

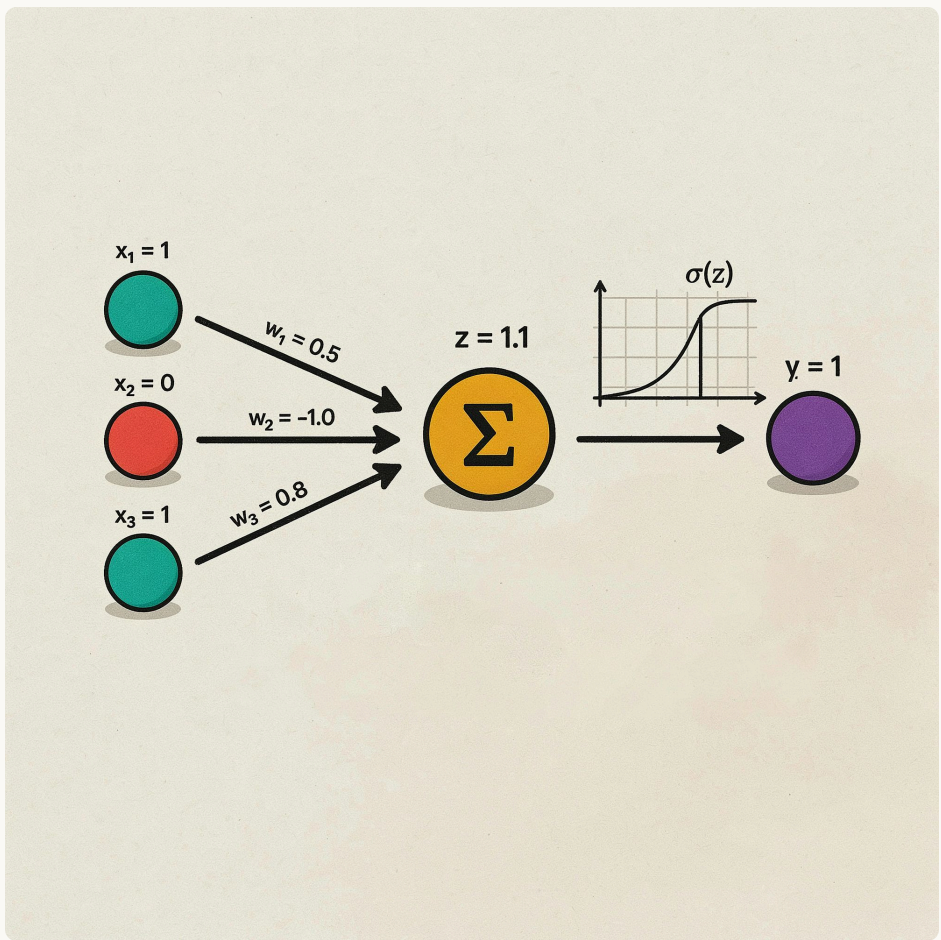
Saída ($\hat{y}$)

O neurônio dispara! A saída é $\hat{y} = 1$.

Interpretação: com esses pesos e entradas, o neurônio ativou — equivalente ao "potencial de ação" no neurônio biológico.

Resumo: Neurônio Artificial em Ação

Componente	Símbolo	Valor
Entradas	x_1, x_2, x_3	1, 0, 1
Pesos	w_1, w_2, w_3	0,5 / -1,0 / 0,8
Bias	b	-0,2
Soma ponderada	z	$0,5 + 0 + 0,8 - 0,2 = 1,1$
Função de ativação	$f(z)$	Degrau
Saída	$\hat{y}$	1 ✓ (neurônio disparou)



Teoria do Perceptron

O Perceptron, proposto por Frank Rosenblatt em 1958, é o modelo mais simples de neurônio artificial com capacidade de aprendizado. Sua teoria formaliza as condições e garantias do processo de treinamento.

Regra de Aprendizagem do Perceptron

$$w_j(n + 1) = w_j(n) + \eta \cdot e(n) \cdot x_j(n)$$

Onde: η = taxa de aprendizagem, $e(n) = y_{\text{real}} - \hat{y}(n)$ = erro na iteração n , x_j = j -ésima entrada

Teorema da Convergência do Perceptron

Se os dados de treinamento forem **linearmente separáveis**, o algoritmo do Perceptron é garantido de convergir em um número finito de iterações, encontrando um hiperplano separador.

Função de Ativação Degrau (Heaviside)

$$\hat{y} = f(z) = \begin{cases} 1 & \text{se } z \geq \theta \\ 0 & \text{se } z < \theta \end{cases}$$

Onde $z = \sum_j w_j x_j + b$ (combinação linear das entradas)

Limitação Fundamental

O Perceptron de camada única **não consegue resolver problemas não linearmente separáveis**, como o XOR. Essa limitação motivou o desenvolvimento do MLP (Multilayer Perceptron).

Porta AND: Treinamento do Perceptron

O perceptron aprende os pesos iterativamente. Iniciamos com $w_1=0$, $w_2=0$ e $b=0$, taxa de aprendizagem $\eta=1$, e aplicamos a regra de atualização a cada amostra.

$$w_j(n+1) = w_j(n) + \eta \cdot x_j \cdot e(n) \quad \text{onde} \quad e(n) = y_{real} - \hat{y}$$

Tabela Verdade AND

x_1	x_2	y_{real}
0	0	0
0	1	0
1	0	0
1	1	1

Estado Inicial

$$w_1 = 0, w_2 = 0, b = 0, \eta = 1$$

01

Amostra (0,0) $\rightarrow y_{real} = 0$

$$z = 0 \cdot 0 + 0 \cdot 0 + 0 = 0 \rightarrow f(z) = 1 \text{ (degrau: } z \geq 0 \rightarrow 1)$$

$$e = 0 - 1 = -1$$

$$w_1=0 \mid w_2=0 \mid b=-1 \checkmark$$

03

Amostra (1,0) $\rightarrow y_{real} = 0$

$$z = 0 \cdot 1 + 0 \cdot 0 + (-1) = -1 \rightarrow f(z) = 0$$

$$e = 0 - 0 = 0 \rightarrow \text{sem atualização}$$

$$w_1=0, w_2=0, b=-1 \text{ (sem mudança)}$$

02

Amostra (0,1) $\rightarrow y_{real} = 0$

$$z = 0 \cdot 0 + 0 \cdot 1 + (-1) = -1 \rightarrow f(z) = 0$$

$$e = 0 - 0 = 0 \rightarrow \text{sem atualização}$$

$$w_1=0, w_2=0, b=-1 \text{ (sem mudança)}$$

04

Amostra (1,1) $\rightarrow y_{real} = 1$

$$z = 0 \cdot 1 + 0 \cdot 1 + (-1) = -1 \rightarrow f(z) = 0$$

$$e = 1 - 0 = 1$$

$$w_1=1 \mid w_2=1 \mid b=0 \checkmark$$

$\rightarrow$ **Pesos ao final da Época 1: $w_1=1, w_2=1, b=0$**

Porta AND: Época 2 e Convergência

Continuando com os pesos obtidos na Época 1: $w_1=1$, $w_2=1$, $b=0$. Verificamos se o perceptron já aprendeu corretamente.

01

Amostra (0,0) $\rightarrow y_{\text{real}} = 0$

$z = 1 \cdot 0 + 1 \cdot 0 + 0 = 0 \rightarrow f(z) = 1$ (degrau: $z \geq 0 \rightarrow 1$)

$e = 0 - 1 = -1$

$w_1 = 1 + 1 \cdot 0 \cdot (-1) = 1 \mid w_2 = 1 + 1 \cdot 0 \cdot (-1) = 1 \mid b = 0 + 1 \cdot (-1) = -1$
(atualização dos pesos)

03

Amostra (1,0) $\rightarrow y_{\text{real}} = 0$

$z = 1 \cdot 1 + 0 \cdot 0 + (-2) = -1 \rightarrow f(z) = 0$

$e = 0 - 0 = 0 \rightarrow$ sem atualização

$w_1 = 1, w_2 = 0, b = -2$

O processo continua...

O perceptron continua iterando até que nenhuma amostra gere erro ($e=0$ para todas).

02

Amostra (0,1) $\rightarrow y_{\text{real}} = 0$

$z = 1 \cdot 0 + 1 \cdot 1 + (-1) = 0 \rightarrow f(z) = 1$ (degrau: $z \geq 0 \rightarrow 1$)

$e = 0 - 1 = -1$

$w_1 = 1 + 1 \cdot 0 \cdot (-1) = 1 \mid w_2 = 1 + 1 \cdot 1 \cdot (-1) = 0 \mid b = -1 + 1 \cdot (-1) = -2$
(atualização dos pesos)

04

Amostra (1,1) $\rightarrow y_{\text{real}} = 1$

$z = 1 \cdot 1 + 0 \cdot 1 + (-2) = -1 \rightarrow f(z) = 0$

$e = 1 - 0 = 1$

$w_1 = 1 + 1 \cdot 1 \cdot 1 = 2 \mid w_2 = 0 + 1 \cdot 1 \cdot 1 = 1 \mid b = -2 + 1 \cdot 1 = -1$
(atualização dos pesos)

$\rightarrow$ **Pesos ao final da Época 2: $w_1=2, w_2=1, b=-1$**

Verificação Final (pesos convergidos)

Com pesos finais convergidos (ex: $w_1=1, w_2=1, b=-1,5$), testar todas as amostras: (0,0): $z=-1,5 \rightarrow 0 \checkmark \mid$ (0,1): $z=-0,5 \rightarrow 0 \checkmark \mid$ (1,0): $z=-0,5 \rightarrow 0 \checkmark \mid$ (1,1): $z=0,5 \rightarrow 1 \checkmark$

Exercício: Treine o Perceptron para a Porta OR

Com base no exemplo da Porta AND, treine o perceptron para implementar a função lógica OR. Use os mesmos parâmetros iniciais: $w_1=0$, $w_2=0$, $b=0$ e taxa de aprendizagem $\eta=1$.

Tabela Verdade OR

x_1	x_2	y_{real}
0	0	0
0	1	1
1	0	1
1	1	1

Estado Inicial

$w_1 = 0, w_2 = 0, b = 0, \eta = 1$

Função de ativação: Degrau

$f(z) = 1$ se $z \geq 0$, senão 0

Passo 1 — Época 1, Amostra (0,0)

Calcule $z = w_1 \cdot 0 + w_2 \cdot 0 + b$. Aplique $f(z)$.
Calcule $e = y_{\text{real}} - \hat{y}$. Atualize w_1 , w_2 e b conforme a regra de aprendizagem.

Passo 2 — Época 1, Amostra (0,1)

Use os pesos atualizados. Calcule $z = w_1 \cdot 0 + w_2 \cdot 1 + b$. Aplique $f(z)$. Calcule e e atualize os pesos.

Passo 3 — Época 1, Amostra (1,0)

Use os pesos atualizados. Calcule $z = w_1 \cdot 1 + w_2 \cdot 0 + b$. Aplique $f(z)$. Calcule e e atualize os pesos.

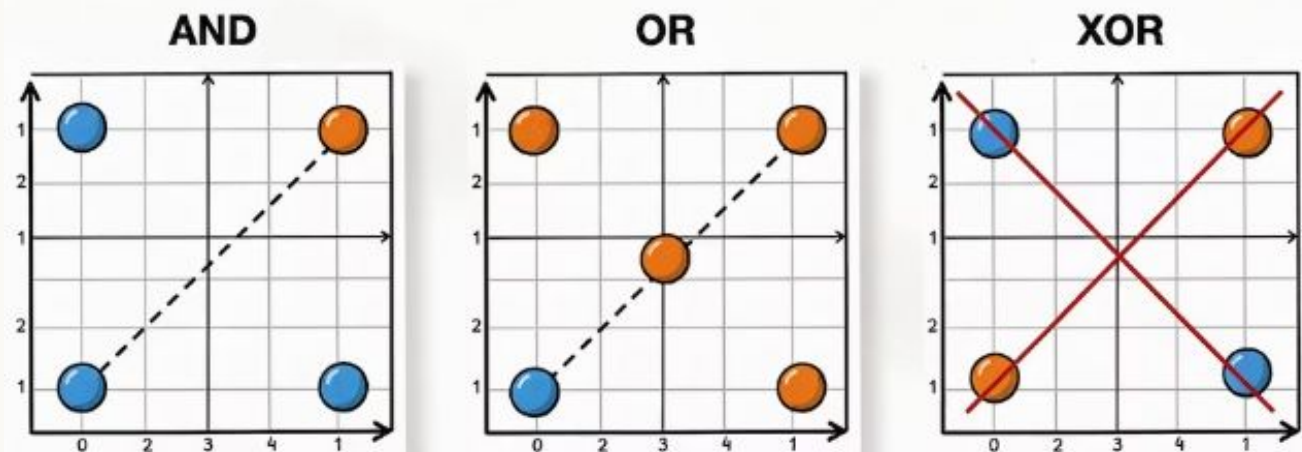
Passo 4 — Época 1, Amostra (1,1)

Use os pesos atualizados. Calcule $z = w_1 \cdot 1 + w_2 \cdot 1 + b$. Aplique $f(z)$.
Calcule e e atualize os pesos. **Quais são os pesos ao final da Época 1?**

Continue até Convergir

Repita o processo para as épocas seguintes. **Dica:** a função OR é linearmente separável — o Teorema da Convergência do Perceptron garante que o algoritmo converge! 🎯

Separabilidade Linear: AND, OR e XOR



AND ✓

Os três pontos com saída 0 ficam de um lado e o único ponto com saída 1 (1,1) do outro. Uma única reta separa as classes.

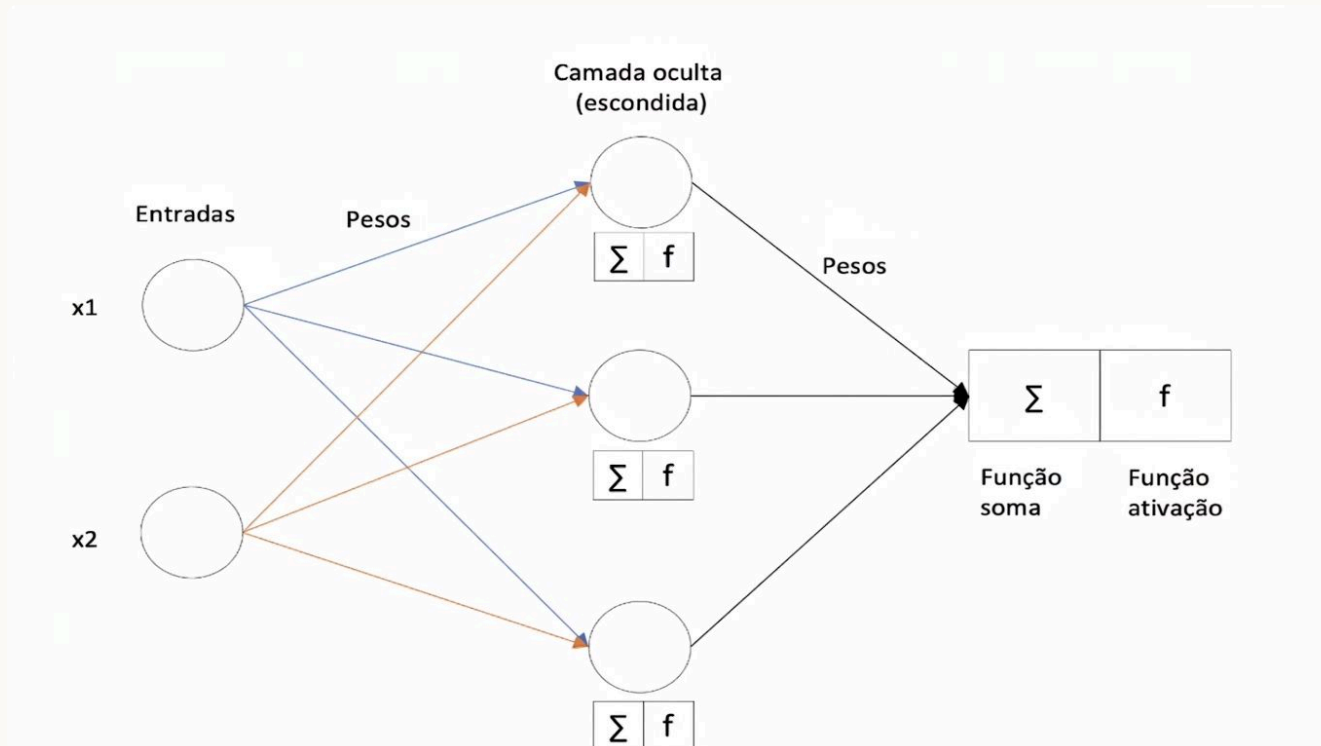
OR ✓

O único ponto com saída 0 (0,0) fica isolado de um lado. Os três pontos com saída 1 ficam do outro. Uma reta resolve.

XOR ✗

Os pontos com saída 1 estão em (0,1) e (1,0) — diagonalmente opostos aos zeros. Nenhuma reta consegue separar as classes. Exige MLP!

Multilayer Perceptron (MLP)



Entradas (x_1, x_2)

Cada entrada é conectada a todos os neurônios da camada oculta por meio de pesos. Cada conexão tem um peso diferente, que é ajustado durante o treinamento.

Camada Oculta (Escondida)

Cada neurônio oculto realiza: soma ponderada das entradas (Σ) seguida de uma função de ativação f . Permite aprender representações não lineares dos dados – o que o perceptron simples não consegue.

$$h_j = f\left(\sum_i w_{ij}x_i + b_j\right)$$

Pesos entre Camadas

Cada camada possui sua própria matriz de pesos. Os pesos são ajustados pelo algoritmo de Backpropagation para minimizar o erro da saída.

MLP: Camada de Saída e Poder de Representação

Continuando a análise do MLP — como a camada de saída processa os sinais e por que múltiplas camadas são essenciais para problemas não lineares.

Camada de Saída

Recebe as saídas da camada oculta (h_1, h_2, h_3), realiza nova soma ponderada (Σ) e aplica função de ativação f para produzir a saída final $\hat{y}$.

$$\hat{y} = f\left(\sum_j w_j h_j + b\right)$$

A função de ativação da saída depende do problema: sigmoide para classificação binária, softmax para multiclasse, linear para regressão.

Por que "Multi" Camadas?

Com pelo menos uma camada oculta, o MLP consegue resolver problemas não linearmente separáveis (como XOR), aproximando qualquer função contínua — Teorema da Aproximação Universal (Cybenko, 1989).

$$\text{MLP} \approx f : \mathbb{R}^n \rightarrow \mathbb{R}^m \quad (\text{qualquer função contínua})$$

Quanto mais camadas e neurônios, maior a capacidade de representação — mas também o risco de overfitting.

Entrada $\rightarrow$ Dados brutos $x_1, x_2, \dots, x_n$

Camada(s) Oculta(s) $\rightarrow$ Transformações não lineares: $h = f(Wx + b)$

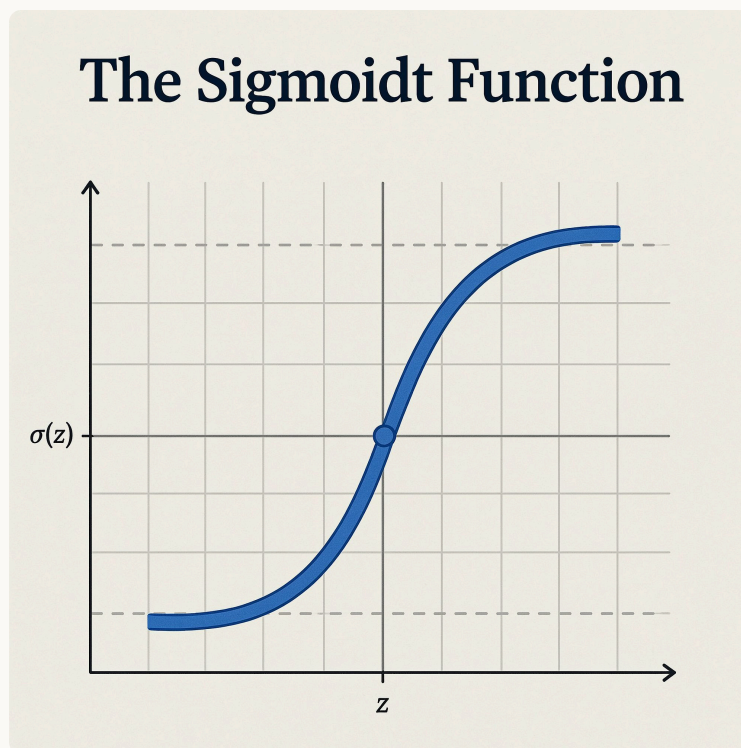
Saída $\rightarrow$ Predição final: $\hat{y} = f(Wh + b)$

A Função Sigmoid: Ativação Essencial do MLP

A função sigmoide foi a função de ativação dominante nas primeiras redes neurais e no MLP clássico. Ela transforma qualquer valor real em uma saída entre 0 e 1, interpretável como probabilidade.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

z	$\sigma(z)$
$-\infty$	0
-2	0,12
0	0,50
2	0,88
$+\infty$	1



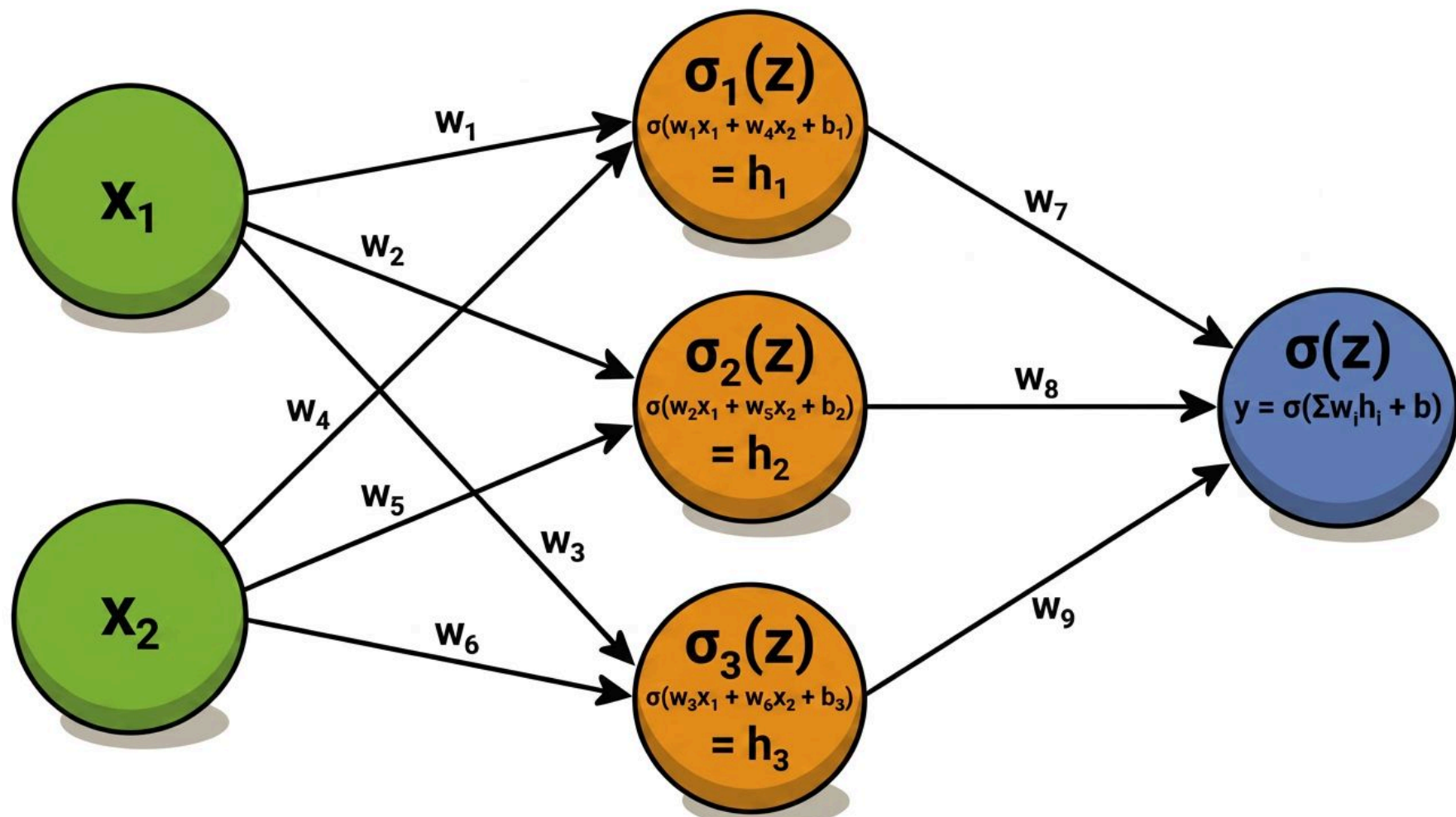
Saída entre 0 e 1

Ideal para classificação binária – interpretável diretamente como probabilidade de pertencer a uma classe.

Não Linearidade

Permite ao MLP aprender fronteiras de decisão complexas – algo impossível com funções lineares.

Arquitetura MLP: 2 Entradas, 3 Neurônios Ocultos, 1 Saída



Arquitetura MLP: Camadas e Propagação Direta

01

MLP

Composição de funções em cadeia:

$f(x) = f^3(f^2(f^1(x)))$. A profundidade é o número de camadas.

02

Camada de Entrada

Distribui os sinais de entrada; não realiza computação.

03

Camadas Ocultas

Atuam como detectores de características, transformando o espaço de entrada. "Ocultas" por não serem diretamente observáveis.

04

Camada de Saída

Gera a resposta final do modelo: rótulo para classificação ou valor contínuo para regressão.

05

Propagação Direta (Forward)

Fluxo de informação unidirecional da entrada para a saída, sem feedback.

06

Cálculo por Camada

$$h = \varphi(W^T x + b)$$

Onde h é a saída da camada, φ a função de ativação, W os pesos, x a entrada e b o bias.

MLP para XOR: Forward Pass com Sigmoide

Considere uma MLP com 2 entradas (x_1, x_2), 3 neurônios na camada oculta e 1 neurônio de saída. Usamos a sigmoide como ativação. Pesos iniciais arbitrários para ilustrar o forward pass da primeira rodada.

Pesos Iniciais (arbitrários)

Camada Oculta (W^1)

	h_1	h_2	h_3
x_1	0,5	0,3	-0,4
x_2	-0,2	0,8	0,6
b	0,1	-0,3	0,2

Camada de Saída (W^2)

	$\hat{y}$
h_1	0,7
h_2	-0,5
h_3	0,4

Exemplo: Amostra $x_1=1, x_2=1$ (XOR=0)

Passo 1 – Camada Oculta:

$$z_{h_1} = 0,5(1) + (-0,2)(1) + 0,1 = 0,4 \Rightarrow h_1 = \sigma(0,4) \approx 0,599$$

$$z_{h_2} = 0,3(1) + 0,8(1) + (-0,3) = 0,8 \Rightarrow h_2 = \sigma(0,8) \approx 0,690$$

$$z_{h_3} = -0,4(1) + 0,6(1) + 0,2 = 0,4 \Rightarrow h_3 = \sigma(0,4) \approx 0,599$$

Passo 2 – Camada de Saída:

$$z_{out} = 0,7(0,599) + (-0,5)(0,690) + 0,4(0,599) + (-0,1)$$

$$z_{out} = 0,419 - 0,345 + 0,240 - 0,1 = 0,214$$

$$\hat{y} = \sigma(0,214) \approx 0,553$$

MLP para XOR: Comparação dos Resultados (1ª Época)

Pesos iniciais aplicados às 4 amostras do XOR – resultados do forward pass:

x_1	x_2	XOR (y_{real})	$h_1=\sigma(z)$	$h_2=\sigma(z)$	$h_3=\sigma(z)$	$\hat{y}=\sigma(z)$	Erro ($y-\hat{y}$)
0	0	0	$\approx 0,525$	$\approx 0,426$	$\approx 0,550$	$\approx 0,539$	$\approx -0,539$
0	1	1	$\approx 0,475$	$\approx 0,622$	$\approx 0,690$	$\approx 0,552$	$\approx +0,448$
1	0	1	$\approx 0,646$	$= 0,500$	$\approx 0,450$	$\approx 0,545$	$\approx +0,455$
1	1	0	$\approx 0,599$	$\approx 0,690$	$\approx 0,599$	$\approx 0,553$	$\approx -0,553$

MLP para XOR: Cálculo do MSE — 1ª Época

Com base nos erros obtidos no forward pass, calculamos o MSE para avaliar o desempenho da rede na primeira época.

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Então, para as 4 amostras do XOR:

$$\text{MSE} = \frac{1}{4} [(-0,539)^2 + (+0,448)^2 + (+0,455)^2 + (-0,553)^2]$$

$$\text{MSE} = \frac{1}{4} [0,2905 + 0,2007 + 0,2070 + 0,3058]$$

$$\text{MSE} = \frac{1}{4} \times 1,0040 \approx 0,2510$$

Interpretação do MSE $\approx 0,25$

$\text{RMSE} = \sqrt{0,25} \approx 0,50$ — a rede erra em média 0,50 por amostra.
Como os valores reais são 0 ou 1, isso equivale a um chute aleatório.
A rede ainda não aprendeu nada útil.

O que acontece a seguir?

O Backpropagation usará esses erros para calcular os gradientes e ajustar os 13 parâmetros (W^1 , W^2 , b^1 , b^2) nas épocas seguintes, reduzindo progressivamente o MSE até convergir.

Capítulo 6

Back-Propagation

A não-linearidade que permite às redes neurais aprender funções complexas.



Backpropagation: As Duas Fases

A backpropagation, desenvolvida em meados dos anos 1980, revolucionou o treinamento de Redes Neurais Artificiais ao resolver o complexo "problema da atribuição de crédito".

1

1. Forward Pass

- Pesos sinápticos fixos.
- Sinal propagado da entrada à saída.
- Geração da predição $\hat{y}$.

2

2. Backward Pass

- Cálculo do erro $e = d - \hat{y}$.
- Erro propagado de volta camada por camada.
- Cálculo do gradiente local δ_j de cada neurônio.

Descida do Gradiente

Minimizar $C(w_1, w_2, \dots, w_n)$ ajustando os pesos na direção oposta ao gradiente.

01

Calcular o Gradiente

Calcular $\partial E / \partial w_j$ para cada peso w_j .

02

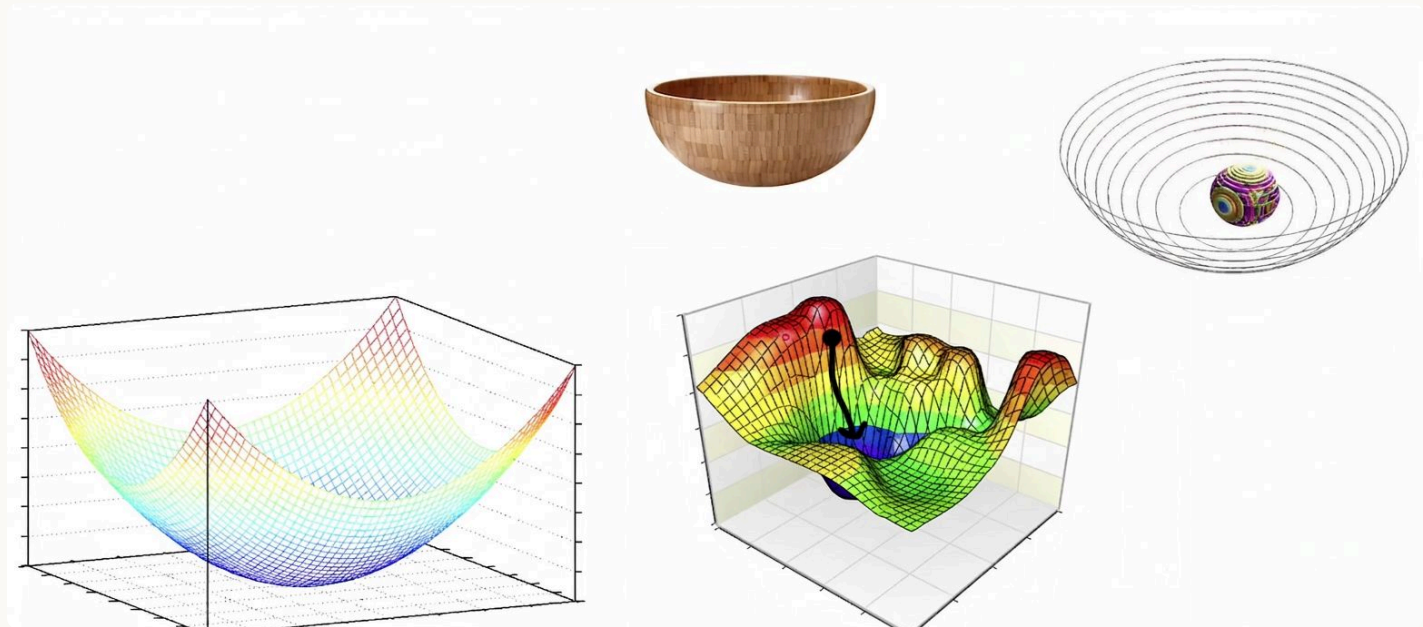
Atualizar os Pesos

Atualizar: $w_j \leftarrow w_j - \eta \cdot (\partial E / \partial w_j)$,
onde η é a taxa de aprendizado.

03

Repetir até Convergir

Repetir até o gradiente se aproximar de zero (mínimo atingido).



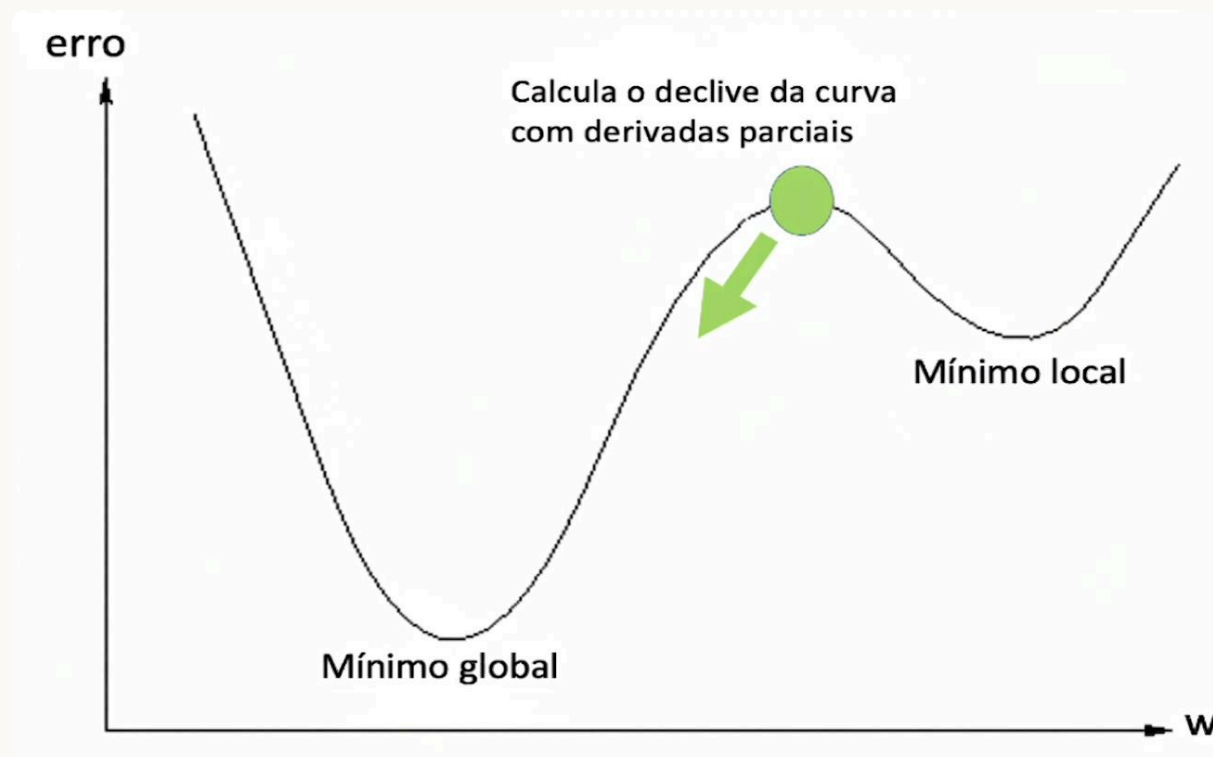
Descida do Gradiente: Mínimo Global vs. Local

Mínimo Global

O ponto mais baixo da função de custo. O objetivo do treinamento é convergir para este ponto.

Mínimo Local

Um vale secundário onde o gradiente é zero, mas não é o menor erro possível. O algoritmo pode ficar preso aqui.



Derivada Parcial: Erro e Pesos

A derivada parcial do erro em relação a cada peso indica a direção e magnitude do ajuste necessário.

O que mede?

$\partial E / \partial w_j$ indica quanto o erro **E** varia ao mudar o peso **w_j** — a sensibilidade do erro a cada peso.

$$\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

O produto das duas derivas em azul é chamado de delta de saída.

Como calcular?

Aplica-se a regra da cadeia:

$\partial E / \partial w_j = \partial E / \partial \hat{y} \cdot \partial \hat{y} / \partial z \cdot \partial z / \partial w_j$, decompondo o gradiente camada a camada.

Passo 1) Cálculo dos Deltas (δ) no Backpropagation

Camada de Saída

Aplicando a regra da cadeia ao MSE: $E = \frac{1}{2}(\hat{y} - y)^2$

$$\frac{\partial E}{\partial \hat{y}} = (\hat{y} - y)$$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$$

$$\delta_{out} = \frac{\partial E}{\partial z} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$$

Termo do erro

$(\hat{y} - y)$: derivada de $\frac{1}{2}(\hat{y} - y)^2$ em relação a $\hat{y}$.

Derivada da sigmoide

$\sigma'(z) = \hat{y} \cdot (1 - \hat{y})$: derivada da função de ativação em relação a z .

Camada Oculta

O delta de cada neurônio oculto k é propagado a partir do δ_{out} ponderado pelo peso de conexão w_k (oculto $\rightarrow$ saída):

$$\delta_{h_k} = \delta_{out} \cdot w_k \cdot h_k(1 - h_k)$$

$$\begin{cases} \delta_{h_1} = \delta_{out} \cdot w_7 \cdot h_1(1 - h_1) \\ \delta_{h_2} = \delta_{out} \cdot w_8 \cdot h_2(1 - h_2) \\ \delta_{h_3} = \delta_{out} \cdot w_9 \cdot h_3(1 - h_3) \end{cases}$$

$\delta_{out} \cdot w_k$

Propaga o erro da saída para o neurônio oculto k , ponderado pelo peso de conexão correspondente (w_7 , w_8 ou w_9).

$h_k \cdot (1 - h_k)$

Derivada local da sigmoide no neurônio oculto k .

Cálculo Simplificado dos Deltas (δ)

Delta da Camada de Saída

$\delta_{saída}$

O erro da saída multiplicado pela derivada da sigmoide naquele ponto:

$$\delta_{saída} = \text{Erro} \cdot \sigma'(\hat{y}) = (y - \hat{y}) \cdot \hat{y}(1 - \hat{y})$$

Erro

$(y - \hat{y})$: diferença entre o valor real e o previsto.

$\sigma'(\hat{y})$

$\hat{y} \cdot (1 - \hat{y})$: derivada da sigmoide na saída.

Delta da Camada Oculta

δ_{oculto}

A derivada da sigmoide no neurônio oculto, multiplicada pelo peso de conexão com a saída e pelo delta da saída:

$$\delta_{h_k} = \sigma'(h_k) \cdot w_k \cdot \delta_{saída} = h_k(1 - h_k) \cdot w_k \cdot \delta_{saída}$$

$\sigma'(h_k)$

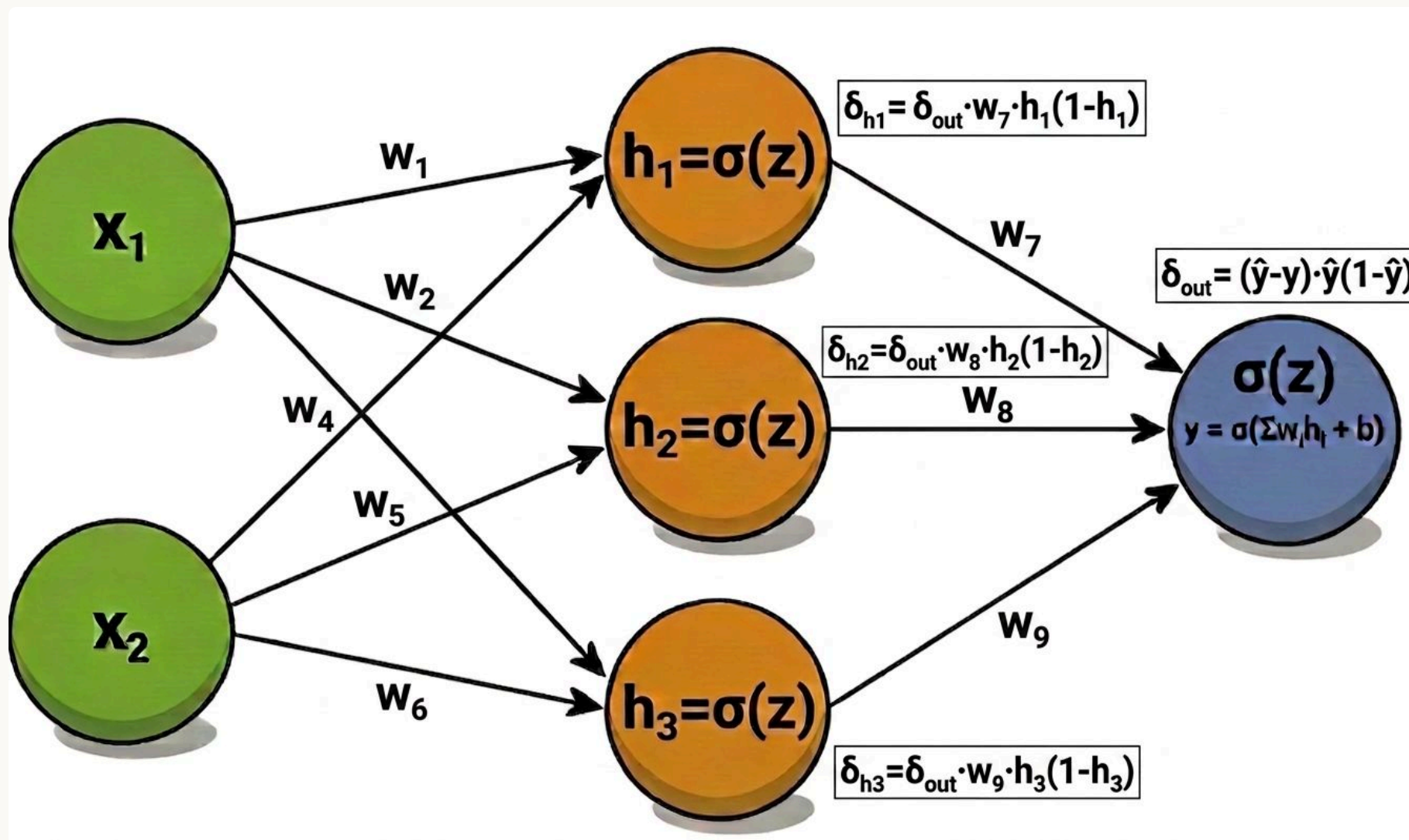
$h_k \cdot (1 - h_k)$: derivada da sigmoide no neurônio oculto k.

$w_k \cdot \delta_{saída}$

Propaga o erro da saída para o neurônio oculto k via peso de conexão w_k .

Após calcular os deltas, a atualização de cada peso é: $w_j \leftarrow w_j + \eta \cdot \delta \cdot x_j$

Deltas no Backpropagation



Passo 2) Atualização dos Pesos

Camada de Saída (W^2)

Cada peso entre a camada oculta e a saída é atualizado usando o $\delta_{saída}$ e a ativação h_k :

$$w_k \leftarrow w_k + \eta \cdot \delta_{saída} \cdot h_k$$

η (eta)

Taxa de aprendizado: controla o tamanho do passo na atualização.

h_k

Ativação do neurônio oculto k: a entrada que chegou ao peso w_k .

Camada Oculta (W^1)

Cada peso entre as entradas e o neurônio oculto k é atualizado usando o δ_{h_k} e a entrada x_j :

$$w_{kj} \leftarrow w_{kj} + \eta \cdot \delta_{h_k} \cdot x_j$$

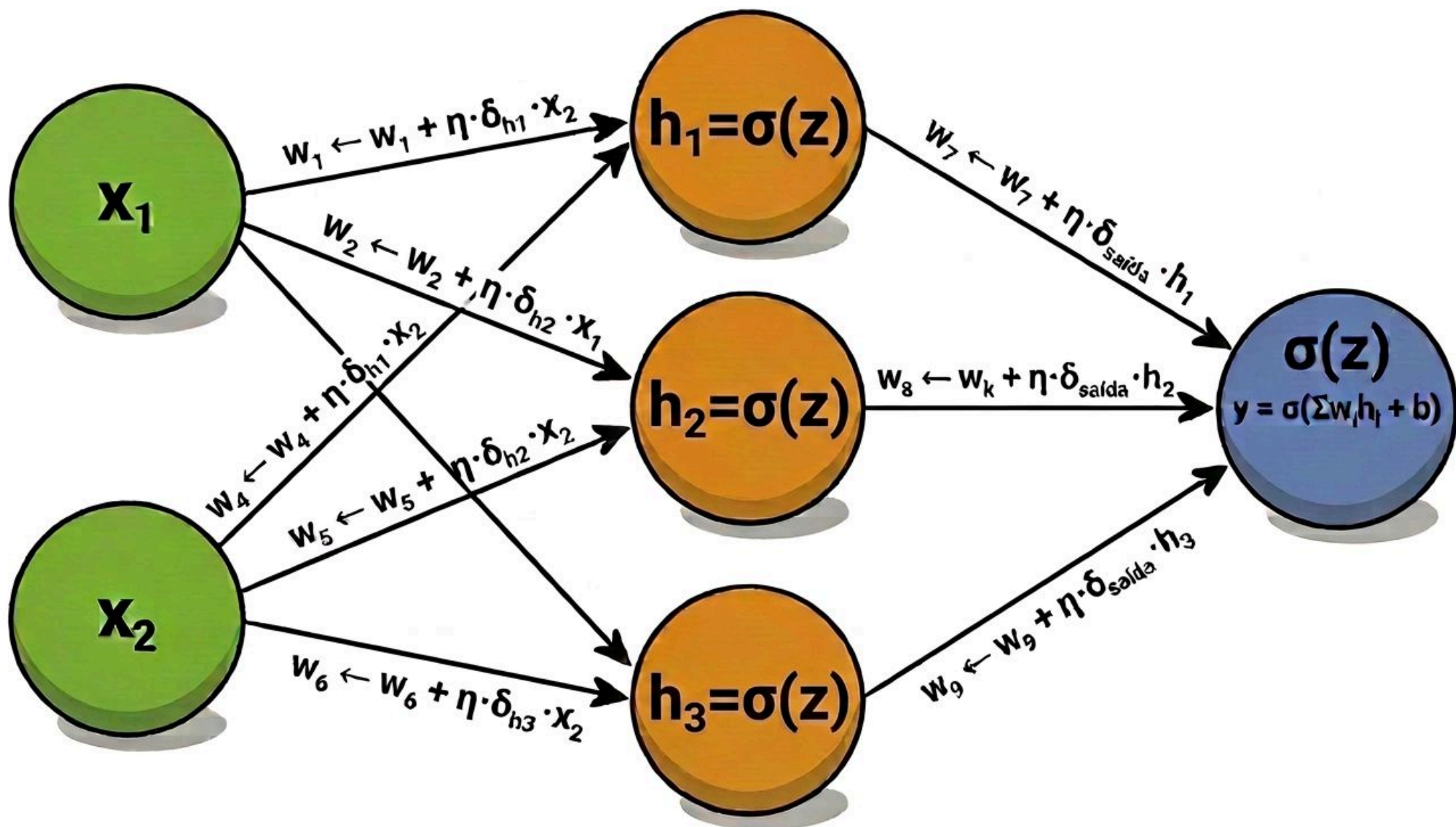
δ_{h_k}

Delta do neurônio oculto k, propagado a partir do $\delta_{saída}$.

x_j

Valor da entrada j: a entrada que chegou ao peso w_{kj} .

O processo repete para todas as amostras e todas as épocas até o erro convergir. O sinal de + assume que δ já carrega o sinal do erro ($y - \hat{y}$).



Otimização: Gradiente Descendente e Variantes

Batch (Lote)

- Atualiza após **VER TODOS** os exemplos.
- Gradiente preciso, mas **lento para grandes datasets**.
- Cácula todos os gradientes, de todos os exemplos → calcula a média dos gradientes → Atualiza todos os pesos

SGD (Estocástico) → O que vimos

- Atualiza após **CADA** exemplo.
- Rápido, mas ruidoso ("**caminhada aleatória**").
- Bom para datasets grandes e redundantes.

Mini-Batch

- Atualiza após **GRUPOS** (32–256 exemplos).
- Padrão moderno. **Eficiente em GPU**.
- Efeito regularizador e estimativa imparcial do gradiente com variância menor.

Taxa de Aprendizado (η): A taxa de aprendizado é crucial – **muito pequena** leva a um treinamento lento (sobreamortecido), enquanto **muito grande** pode causar oscilações e divergência. A técnica de Annealing começa com uma taxa alta e a diminui gradualmente ao longo do treinamento para otimizar a convergência.

Referências Bibliográficas

- HAYKIN, S. Neural Networks and Learning Machines. 3ª ed. Pearson.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. Deep Learning. MIT Press, 2016.
- MITCHELL, T. Machine Learning. McGraw-Hill, 1997.
- BISHOP, C. Pattern Recognition and Machine Learning. Springer, 2006.

Material baseado na apostila: Inteligência Computacional – Fundamentos, Algoritmos e Aplicações em Engenharia Elétrica (Versão Editorial 2.0) – Prof. Dr. Guilherme Augusto Defalque – PPGEE/UFMS